从 Vibe Coding 到 AI 原生研发团队：一套能落地的工程实践

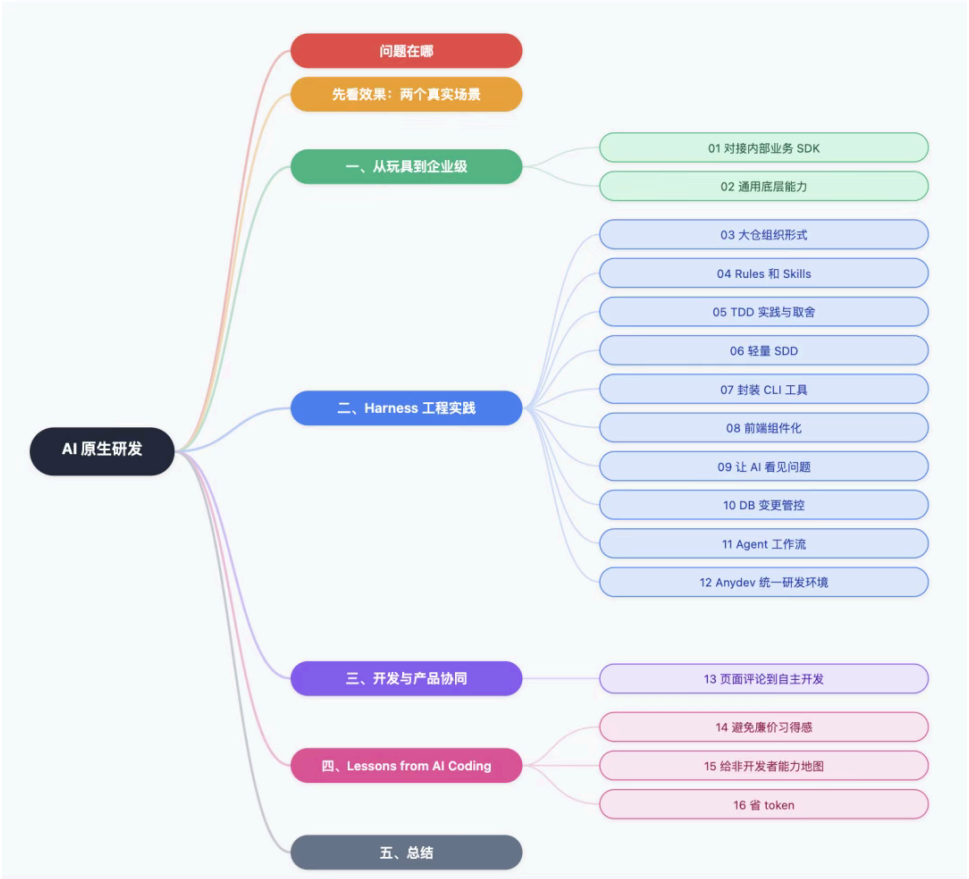
原创 腾讯程序员 腾讯技术工程 Jul 21, 2026, 3:03 AM 广东 747 listened



作者：masoncai

> **导语：** AI 写个人玩具项目很顺手，但落到真实业务里就是另一回事了。这篇整理了我们在打造 AI 原生研发团队上摸出来的一些经验，主要讲四块：通用能力怎么搭、Harness 工程实践、开发跟产品怎么协同。我们希望打破职能边界，让不懂代码的产品同学也能借 AI 东风，自主定义并实现各类内部运营工具，从而带动整个团队的研发效能迈上新台阶。 还有踩完坑之后的一些 Lessons。希望能给大家点参考，尤其是正在琢磨怎么用 AI 把团队能力真正提升起来的同学。

整体概览



让 AI 连续跑一整天，代表了 Harness Engineering 所能达到的一种能力上限。但这种方式更适合从零起步或与现有业务关联度较低的任务，不仅 Token 成本高，日常开发中也很少用到。

日常开发更多是在现有项目中增加功能、修复 Bug，没有必要上那么重的自动化工程，轻量的 Vibe Coding 就足够了。因此，本文不讨论如何把 AI Coding 的能力上限推得更高，而是重点解决一个更日常的问题：**如何守住这类需求的质量下限，同时提高 Token 效率。**

圈子里的各种概念层出不穷——Prompt Engineering、Harness、Agent Loop——各有各的用武之地。事实上，我自己做的很多事情，本质上也是在搭护栏（Harness）。

但说实话，团队里的每个人并不需要一上来就理解并应用这些概念。代码本身就是描述业务逻辑最精确、最高效的语言。对大多数人来说，更实际的起点，是先学会高质量地使用 AI 完成编码任务。

> 真正 ROI 最高、见效最快的方式，是先让 AI Coding 运转起来，再让护栏随着实际需求逐步生长出来。

文中涉及的项目叫 Vibe Flowing，是一个业务属性很强的“AI 原生海外网络运营平台”，主要服务于海外网络运营相关的业务场景。

所谓“AI 原生研发”，在我们这里指的是：项目的所有组件和能力，包括页面功能、定时任务、Agent 智能体、开放 API、外部接口对接等，都由 AI 统一维护。

使用者只需要描述需求。简单需求可以通过对话直接完成；复杂需求则先写成文档，经过研发流程对齐后再进行开发。

在这套机制下，团队里不管会不会写代码，每个人都能参与进来。网络运营同事即使不是开发人员，也能完成**提需求、写代码、做验收**的完整流程。

> 这正是 AI 赋能团队最直观的体现：当门槛降得足够低，人的参与面自然就打开了。

问题在哪



在聊我们的做法之前，先说说为什么要这么做。

VibeCoding 的门槛已经很低了。网络运营同事用各类 AI 编码平台，自己就能写个网页，查查数据、画个图表，满足一时之需。但用着用着问题就来了：

1、重复造轮子、数据不互通、只有自己说得清

每个人写的网页都在各自调用后端系统接口：运营的、网管的、CMDB 的、SNMP 的，各写各的，互不知道。同样是查一条专线的信息，张三的页面查一遍，李四的页面又查一遍，鉴权方式、数据口径还不一样。更麻烦的是数据散落在各个孤岛里，没法关联分析。比如专线流量异常了，想同时看流量趋势、丢包率、运营商质量，得开三四个页面来回切，数据还对不上。

有人尝试把这些页面聚合到一个入口里，放个超链接列表方便大家跳转。但说实话，这有点草台班子：**链接越来越多，页面风格五花八门，哪个是最新的、哪个还能用、哪个出了问题找谁，谁也说不清。时间一长就成了"历史遗产"，没人敢动也没人想动。**

2、想正经做点事，拦路虎太多

运营同事真想做一个能持续用的系统，第一道坎就是各种**系统接口权限的申请**。网管系统要开权限，CMDB 要开权限，数据库要开账号，每一个都是流程、审批、等待。好不容易权限拿到了，**接口文档不全、字段含义不清、鉴权方式各异，AI 想帮忙对接也经常踩坑。出问题了让 AI 排查，一通操作猛如虎，token 烧了不少，往往还没问到点子上。**

> 因为这些底层问题的复杂性远超业务逻辑本身，AI 在没有上下文的情况下很难高效定位。

说到底，Vibe Coding 适合写"一次性"的东西，但做不了"能持续迭代的系统"。缺的不是 AI 的能力，而是一个**专业开发先搭好的架子**：日志、鉴权、权限控制、外部接口封装、数据

库变更流程、定时任务等，都得先打通。有了这个底座，AI 在上面开发就不必每次从零开始，也不在基础设施问题上浪费时间和 token。

这就是做 VibeFlowing 的出发点：**先搭好企业级底座，再让 AI 在上面持续开发，把整个团队的能力边界往外推。**

> 不只是专业开发干活更快，非开发人员也能参与建设真正有用的业务系统。

效果概览



展开讲怎么做之前，先看两个实际跑通的场景，感受一下"AI 原研发"到底是什么体验。

场景一：AI 原生的研发整体流程

运营同事想加一个新功能，比如"出口流量按 AS 聚合的桑基图"，他不需要写需求文档，不需要找开发排期，自己就能搞定。

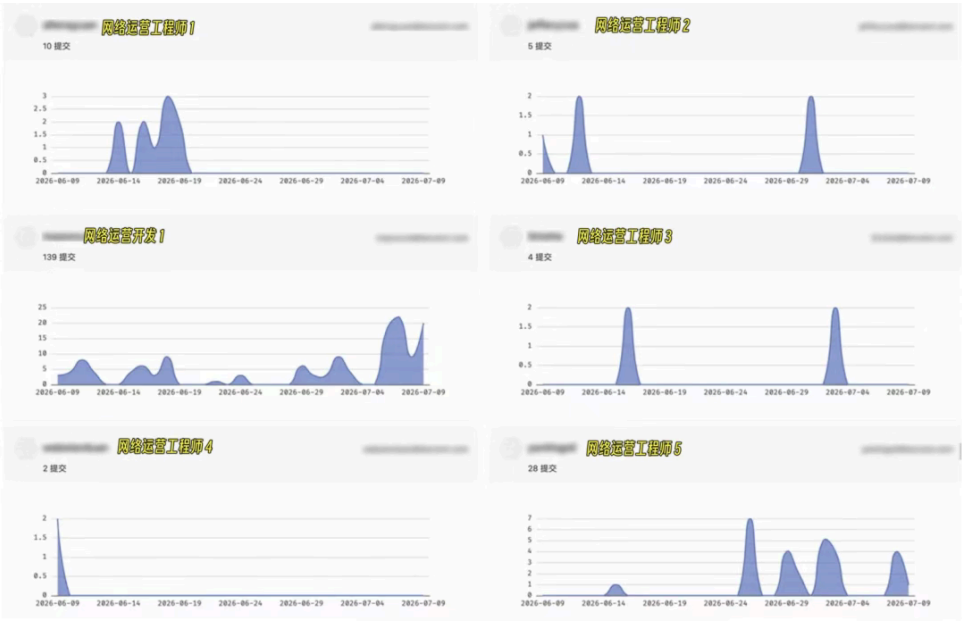
首先，从模板创建一个 anydev 开发容器，**不需要手动配置任何东西，打开 CodeBuddy 后直接告诉 AI 需求**："我想在流量分析页加一个出口流量按 AS 聚合的桑基图，能看到 Top 20 AS 的流量分布。"AI 先和他对齐需求：要展示哪些维度、数据从哪来、放在页面哪个位置，用白话聊几句就确认了。

然后 AI 开始开发。后端接口、前端组件、数据库查询，全栈一把梭。**开发过程中 AI 自己启动开发服务、自己跑测试、自己查日志排错，不需要人介入。**开发完成后，AI 告诉运营同事开发服务的访问地址，让他自己在浏览器里验证。

运营同事打开链接，看到效果满意了，回一句“OK”。AI 自动跑完静态检查和兜底测试，推送代码到远程分支，创建 MR，等管理员审批合并到统一环境（正在引入 AI 审批）。**在合并之前，运营同事在自己的开发容器上就能正常使用这个功能，不用等流水线跑完。**

整个过程，**运营同事没有写一行代码，没有碰过 git，甚至不需要知道"分支"这个概念。**

下图是 Vibe Flowing 项目仓库提交记录，可以看到，架子搭好之后，整个团队都能接住 AI 来实现自己的需求。



这套 AI 原生的开发体系已经在海外网络运营、控制器运营团队应，团队基于统一的 AI 研发基础设施，能够在 3 分钟内全自动完成环境配置，人人可以参与开发。



场景二：AI 原生的 Agent 开发流程

运营事想做一个专线质量分析智能体，按照已有 Agent 平台配置开发的方式，得**等其他研发同学把 MCP 或者 Skills 开放出来，然后他得自己写提示词、配工具，还得自己跑验证，整个过程非常费时间。**

现在在这套体系里，他只需要跟 AI 说：**"我想要一个能分析专线质量的 Agent，输入专线 ID，它自动查流量趋势、丢包率、时延数据，给出质量评估结论和异常原因。验收标准是：随便选一条专线，它能在 30 秒内输出结构化的分析报告。"**

AI 拿到这个目标后，自己去代码仓库里探索：

- 看看现有的数据模型有哪些
- 有哪些 service 可以复用
- 哪些工具函数能直接用

然后自己写系统提示词，定义 Agent 的行为规范和输出格式；自己封装工具函数，把数据查询能力接进来；自己注册到 Agent 框架里。

接下来是调式，AI 自己跑端到端验证，选一条专线试一下，看输出效果。不满足验收标准，就调提示词、换模型、改工具返回格式，反复迭代。

整个 Agent 的开发过程 AI 自己跟自己对话、自己验收，不需要人盯着。

达到效果后，AI 交给运营同事一个能用的智能体。运营同事到网页上的 Agent 聊天页，选这个专线质量 Agent，直接就能用。

下面是一个例子：

1、自然语言描述需求：



2、AI 编码实现并进行自主测试迭代

flow > agent > metro_analysis.py > ...

```python
"""城域网分析 Agent - 分析海外城域网链路组利用率、带宽配置和容量趋势。"""

from langchain_core.tools import BaseTool

from flow.agent.prompts.metro_analysis import METRO_ANALYSIS
from flow.agent._runtime.react_agent import ReactAgent
from flow.agent.tools.metro_analysis import get_metro_analysis_tools


class MetroAnalysisAgent(ReactAgent):
    """城域网分析 Agent。"""

    default_model = "hy-3"
    default_temperature = 0.2

    @property
    def name(self) -> str:
        return "metro_analysis"

    @property
    def title(self) -> str:
        return "城域网分析智能体"

    @property
    def description(self) -> str:
        return "海外城域网分析专家，分析链路组利用率、带宽配置、容量趋势，支持生成专线拓扑图。"

    @property
    def labels(self) -> list[str]:
        return ["城域网"]

    @property
    def system_prompt(self) -> str:
        return METRO_ANALYSIS

    @property
    def tools(self) -> list[BaseTool]:
        return get_metro_analysis_tools()

    @property
    def suggest_questions(self) -> list[dict]:
        return [
            {"text": "帮我统计香港BR到TIX的总带宽"},
            {"text": "查一下香港沙田QCD到葵涌QCDR，今天相比7天前，峰值利用率增加了多少"},
            {"text": "帮我分析新加坡地区，有哪些城域网链路组需要扩容"},
            {"text": "绘制香港B1-BR到葵涌QCDR的专线拓扑"},
        ]
```

3、Agnt 对话效果：



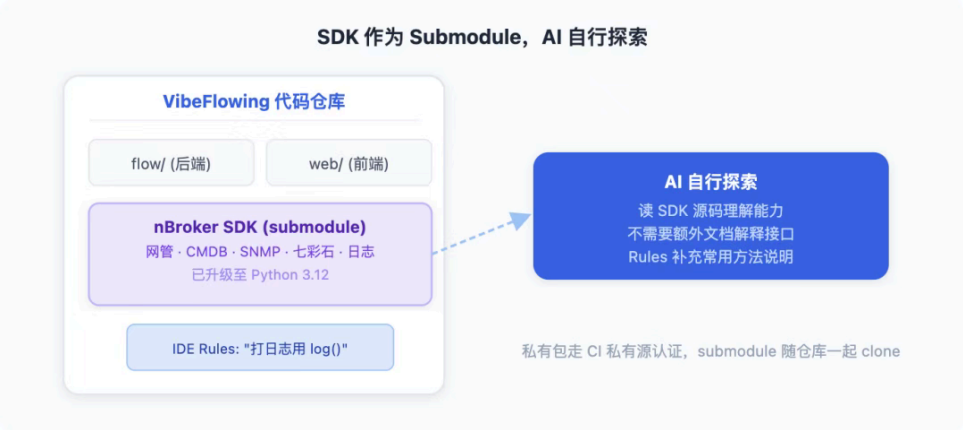
小结

这两个场景的共同点是：人只管定义要什么"，AI 负责"怎么做"。这就是我们理解的 AI 原生研发：**不是一个人的提效工具，而是一个团队的新工作方式。**

接下来，我们来看看具体实践是怎样的。

从玩具到企业级

1、对接内部业务 SDK



网平内部运营系统有很多接口，此前我们已经开发了 Python 本的业务 SDK，封装了各类第三方接口，比如网管系统、CMDB、SNMP 乃至七彩石、日志等能力，在 Vibe 出来的项目中，首先要考虑的是集成这类 SDK。

支撑现网运行的 SDK 此前固定了仅支持 python 3.6.8，去年我们做过升级到 Python 3.12 的探索，将基础依赖都进行了升级，可以直接在新项目中引入，如果是老项目，则需要提前处理好依赖的兼容性问题。

由于此 SDK 是私有包，需要在 CI 流水线中配置私有源认证。为了让 Agent 能理解 SDK 中具备的能力，将该 SDK 作为 sub module 的方式加入到代码仓库，实际编码时，让 Agent 自行探索即可，不需要额外写文档去解释每个接口。

对于一些日志、配置获取等基础常用能力，在 IDE 的 Rules 里写几行说明就行，比如"打日志用 `from nBroker.lib.logger import log` "，AI 记住后每次写代码都会用对。

2、通用底层能力



一个企业级运营系统应该具备几个通用能力，包括：

- 日志
- 页面权限控制
- 页面访问审计
- 开放 API
- MCP 工具
- 定时任务
- 工作流

这些能力每写一个项目都从头搭，成本太高。我们把它沉淀成一套标准设施，新项目直接继承。下面逐个说怎么做的。

日志：直接复用 SDK 中的统一日志方法，全项目统一调用签名，不自己造轮子。AI 写代码时只需要在 Rules 里写一行说明就够了，不需要每次都教。

页面权限控制：我们实现了 RBAC 权限模型。用户的身份信息由太湖网关注入，后端解码后拿到 login_name，再从人事系统查到部门和组。权限规则存在数据库里，支持按页面和操作两个粒度控制，管理员列表走七彩石远程配置，方便动态调整。这套机制对 AI 透明，AI 写新页面时，只需要声明是否受限资源，权限校验自动生效。

页面访问审计：所有请求都经过鉴权中间件，用户身份、访问路径、API Token 使用记录都落库留痕，方便审计追踪。

开放 API：外部系统集成走 API Token 机制，支持 Bearer Token 和自定义 Header 两种方式。每个 Token 可以配置允许访问的路径白名单、管理员列表、过期时间。Token 明文仅管理员可见，其他人看到的是掩码。这套机制让外部系统接入安全可控，AI 也能自助创建和管理 Token。

MCP 工具：基于 FastMCP 搭建了 MCP Server，把专线分析、拓扑分析等核心能力封装成 MCP 工具，供其他 AI 客户端调用。MCP 路径在鉴权白名单中，方便外部集成。

定时任务：基于 APScheduler 实现了装饰器注册机制，新增定时任务只需写一个 `@cron_job` 装饰器函数，声明任务 ID、执行间隔和描述即可。定时任务独立进程运行，不阻

塞 API。支持前端页面暂停/恢复/修改间隔/手动触发，多副本环境下通过数据库抢占保证一条指令只执行一次。AI 新增定时任务时，只需要写业务函数，调度和管理的"脚手架"自动就位。

工作流：基于 DBOS 实现了简单的工作流调度能力，让平台能够支撑复杂流程的运行，这些流程中间可以有人工待办、异步回调等待，从而让一个复杂的任务能够持续跑一个月甚至更长时间而不间断，即使中断了也能从历史状态恢复，即所谓的 Durable Function。

小平台内置这些能力，就意味着 AI 能够帮我们把这些都搞定，而不需要跨系统交互，这是 AI 原生研发的基础条件。

Harness工程实践

1、大仓组织形式



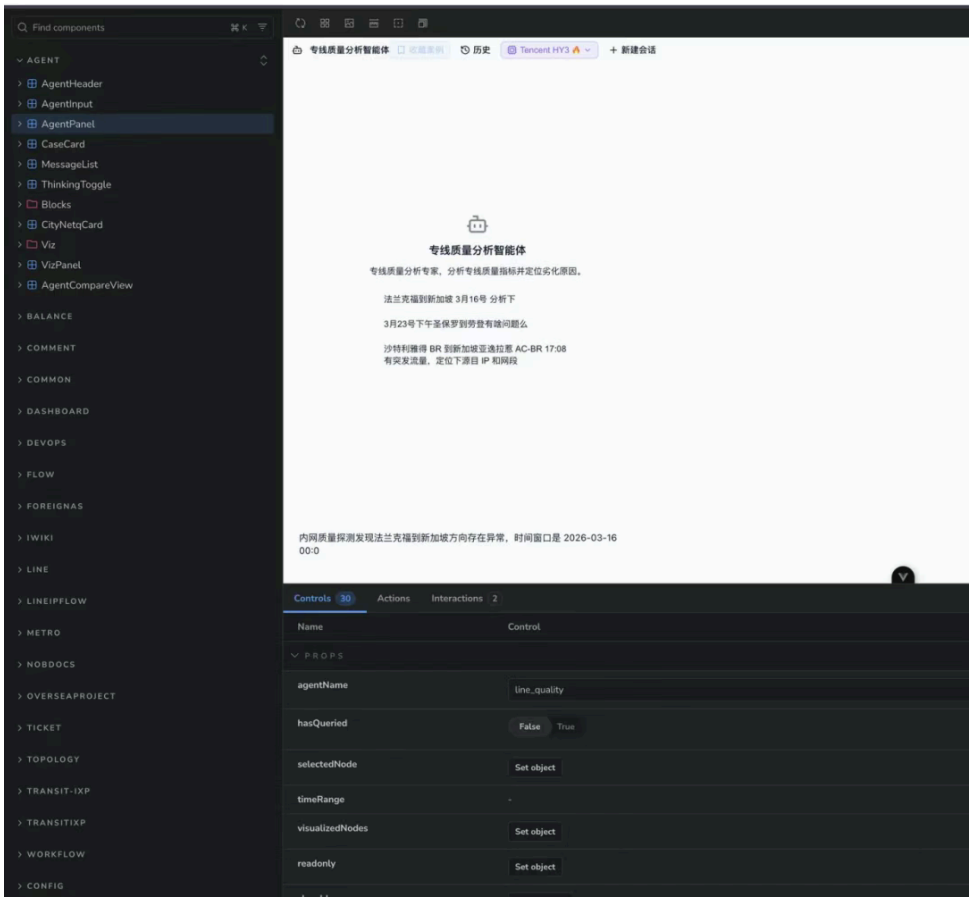
我们采用单仓 monorepo 的形式，后端 `flo/` 和前端 `web/` 在同一个 Git 仓库里。好处是 AI 在一次会话中可以同时看到前后端代码，做全栈改动时上下文完整，不需要跨仓库切换。

后端的目录结构遵循严格的分层约定：controllers（API 层）→ services（业务层）→ models（数据层）→ source（外部接口），调用方向只能向下，禁止反向依赖或同层互调。此外还有 analysis（数据分析）、cron（定时任务）、cli（命令行工具）、agent（智能体）等独立目录。这个约束写在 AGENTS.md 里，AI 每次写代码都会遵守。

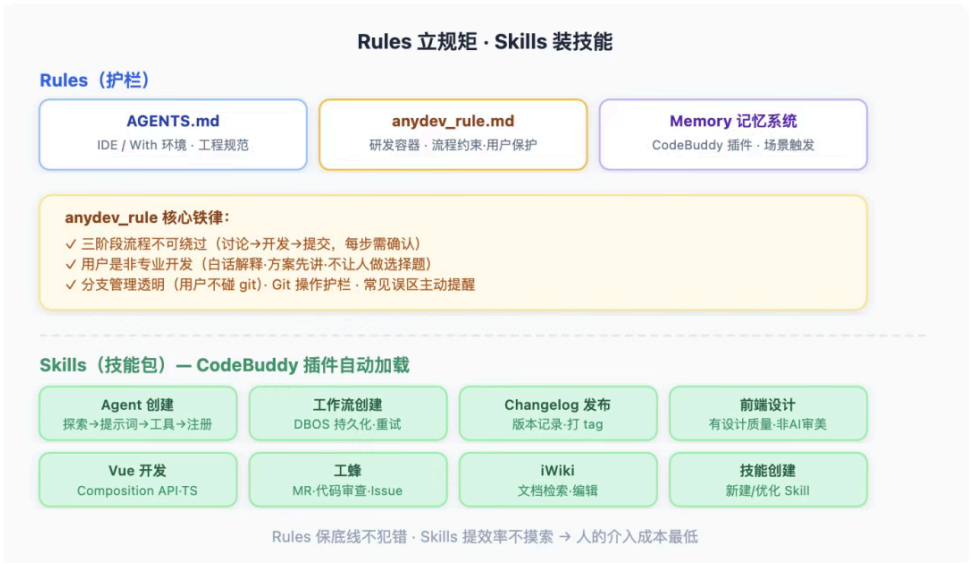
关键设计是每个职能目录下都有一个 `_framework/` 子目录，存放框架级的"脚手架"代码。业务代码只管写业务函数，框架代码负责调度和生命周期管理。这样 AI 写新功能时聚焦业务逻辑，不会被基础设施的细节干扰。

前端同理，`components/` 按业务域分目录，每个组件配套 Storybook story 文件。`views/` 只做组件协调，保持整洁。这套结构符合直觉，AI 探索代码库时能快速定位，人也容易审查。

下面是前端组件化示例，可以看到积累了相当多可复用的组件，这些组件主要是按照业务模块划分：



2、用 Rules 和 Skills 给 AI 立规矩、装技能



大仓结构解决"代码怎么组织"，Rules 和 Skills 解决"AI 怎么干活"。这一环很关键，**不做约束，AI 每次都是"自由发挥"，质量全看运气；做了约束，行为就有了下限保障。**

Rules：分场景加载的项目护栏

我们设计了多层 Rules，按场景自动加载到 AI 的上下文中：

第一层是 **AGENTS.md**，放在项目根目录。这是面向 CodeBuddy IDE 和 With 开发环境的通用规则文件，集中了所有开发护栏和工程偏好——文件红线、后端分层规范、前端工程约定、DB 变更流程、流量图表配色、网络拓扑可视化偏好等。AI 每次会话都会读这个文件，相当于随身带着一本"项目规范手册"。

第二层是 `.vscode/anydev_rule.md`，这是面向统一研发容器的规则文件，会自动加载到每个开发同事的容器环境中。相比 AGENTS.md 侧重工程规范，anydev_rule 更侧重**研发流程约束和用户保护**，核心是几条铁律：

- **三阶段流程不可绕过**：任何需求，哪怕改一个文案，都必须走"**需求讨论** → **开发实现** → **确认提交**"三阶段，每个阶段之间需要用户明确确认，禁止 AI 自行判断"小改动可以跳过讨论"
- **用户是非专业开发**：出现技术名词必须用白话先解释，方案先讲再动手，不让用户做选择题，拿不准就停下来问
- **分支管理对用户透明**：用户不需要关心分支，AI 全程管理（自动切到 `dev-$用户名` 分支、自动同步远程 dev、自动处理冲突）
- **Git 操作护栏**：禁止破坏性命令，改动范围最小化，删除文件前必须确认
- **产品同学常见误区主动提醒**：比如用户说"把数据清掉"，AI 要主动告知 anydev 账号没有 DELETE 权限，建议改用软删除

第三层是 **CodeBuddy 内网版插件的记忆系统**。项目规则中有一部分以 Memory 形式存在，比如"数据库时间字段统一用 DATETIME"、"前端代码修改后必须跑 type-check"、"流量图表入流量绿色、出流量蓝色"等。这些记忆会在 AI 相关场景自动触发，不需要每次重复说明。

三层 Rules 叠加，基本覆盖了"AI 在这个项目里什么能做、不能做、怎么做"的全部约束。设计原则就一条：**规则集中、分场景加载、不重复**。AI 上下文有限，规则散乱或重复既费 token 又容易让 AI 混淆。

关键 Rule：Anydev 云开发护栏以及开发示例如下，左侧文件目录中可以看到有 4 个关键开发约束，右侧是 AI 完成开发后交给人验证时回复的效果：



Skills：给 AI 预装"专业技能包"

Rules 管的是"规矩"，Skills 管的是"能力"。我们沉淀了一批常用 Skill，覆盖项目开发的高频场景：

- **Agent 创建**：新增 ReAct Agent 的完整工作流，探索代码、写提示词、创建工具、注册、验证
- **工作流创建**：基于 DBOS 的工作流编排，处理需要持久化和重试的复杂任务
- **Changelog 发布**：生成 changelog 条目并创建 git tag，处理版本发布
- **前端设计**：创建有设计质量的前端界面，避免"AI 审美"的通用感
- **Vue 开发**：Vue3 + Composition API + TypeScript 的开发规范和最佳实践
- **工蜂**：代码平台操作，仓库管理、合并请求、代码审查、Issue 管理
- **代码腐化处理**：这个特别重要，AI 原生研发跑起来之后，代码仓库的演进速度会特别快，但硬币的另一面是代码腐化也来得更快，我们创建了 2 个职责分离的代码去腐化技能，扫描代码问题、创建 issue，以及认领 issue、修复代码问题，避免“既当裁判又当运动员”。让 Agent 高频小批量定期运行这两个技能，保障仓库代码质量
- **iWiki**：企业内部文档的检索和编辑
- **技能创建**：创建新的 Skill 或优化已有 Skill

这些 Skill 在 CodeBuddy 内网版插件中会自动加载。AI 在遇到对应场景时自动触发，不需要人手动调用。比如用户说"新增一个专线质量分析 Agent"，Agent 创建 Skill 就会自动激活，AI 按照 Skill 里定义的工作流一步步执行：先探索现有代码理解数据模型，再写提示词和工具，然后注册和验证。

Rules 和 Skills 的关系：**Rules 保底线（不犯错），Skills 提效率（干活快）**。光有 Rules，AI 不犯错但每次得摸索怎么干；光有 Skills，AI 干得快但可能不合规范。两者一结合，AI 既守规矩又高效，人的介入成本就降到最低。

3、TDD 实践与取舍



TDD（测试驱动开发）在 AI 编码场景下有一个微妙的问题：AI 写测试很快，但也很容易写出"自欺欺人"的测试，例如只覆盖 happy path，或者断言太弱。我们的做法是用规则约束来兜底，而非追求严格的"先写测试"。

后端用 pytest，前端用 vitest 做组件测试、Playwright 做 E2E。提交前必须通过静态检查：后端跑 ruff（代码风格）+ ty check（类型检查），前端跑 oxlint + vue-tsc。这些检查命令都写在 AGENTS.md 里，AI 会自觉执行。

关于"前端功能测试能否覆盖后端"这个问题，我们的经验是：前端 E2E 测试能验证整条链路（前端 → API → DB），对于业务逻辑的正确性保障是有价值的。但后端单元测试的价值在于快速定位——E2E 挂了你不知道是前端渲染问题还是后端逻辑问题。所以我们的取舍是：核心业务逻辑必须有后端单元测试，前端 E2E 主要覆盖关键用户流程，两者互补而非替代。

覆盖率目标定在 70%，作为参考值不阻断。重要的是改动的文件要有测试覆盖，而不是盲目追求数字。

实际操作中，我们更看重"验收"而非"测试"。对于网络运营同事提的需求，最终验收标准是"页面表现符合预期"，这个验收过程本身就是在做端到端验证。AI 开发完一个功能后，用 Playwright headless 截图验证，再交由提需求的人确认，形成闭环。

4、轻量 SDD



SDD（Spec Driven Development，规格驱动开发）听起来很重，大家一般会使用各类开源 Spec 框架来保障整体流程，但我们用一套极简的文件命名约定就实现了类似的效果。

核心思路：需求文档就是开发的唯一输入。简单的需求直接对话，复杂的写成 Markdown 文档，AI 读完文档后讨论、开发、验收。

文档流转规则：

- `features/` 目录存放需求文档，按 `draft_` → `ready_` → `done/` 三阶段流转
- `draft_` 前缀：未确定的方案，AI 和人一起讨论迭代
- `ready_` 前缀：方案对齐确认，可以开始开发
- 开发完成后移到 `done/` 目录

方案讨论文档放在 `ai_docs/running/` 目录，同样遵循 `draft_` → `ready_` 的命名约定，完成后移到 `ai_docs/done/` 。

这套机制的好处是：AI 一次会话中读完一个 `ready_` 文档就拥有了完整的需求上下文，不需要人来反复解释。而文档本身也是 AI 协助写的，运营同事口述需求，AI 整理成结构化文档，开发同事 review 确认后，AI 改前缀为 `ready_` ，并开始开发。

一句话：**文档命名约定就是工作流**。不用额外的项目管理工具，文件名本身就在表达状态。

5、封装 CLI 工具



AI 在开发过程中经常需要做一些"运营性"操作：执行 DDL、回填数据、管理配置。如果让 AI 自己写临时脚本去 import db 跑，既不安全也不留痕。我们给 AI 封装了几个 CLI 工具，让这些操作有规可循。

flow-db-exec：数据库变更执行工具，是 DB 变更的唯一入口。支持单条 SQL 执行和按文件行号片段执行两种方式，还有 dry-run 干跑模式。这个工具做了三件事：一是高危关键字（DROP/DELETE/TRUNCATE）硬拦截，和数据库账号权限互为双保险；二是强制留痕，SQL 必须先写到 changelog.sql 再执行；三是执行结果清晰可读，SELECT 结果格式化输出，写操作返回受影响行数。

flow-config：业务配置管理工具，支持增删改查和按前缀过滤，配置存在数据库里，方便运行时动态调整。多说一句：很多团队习惯把所有配置丢到远程配置中心去管，但实际用下来会发现，评审发布流程比较繁琐，配置和代码分属两个系统，割裂感强，AI 也很难直接操作。其实大部分业务配置，例如阈值、开关、参数等，没那么敏感，没必要搞那么重。我们直接用项目内部的数据库表管，配一个 CLI 加一个管理页面就够了。配置和代码同在一个仓库，AI 能直接读写，改完即生效，不用跨系统走流程。Key 按模块分层命名（如 `threshold.surge_ratio`），同模块配置聚合成 dict 读取，代码里统一走 `app_config` 模型，不直接拼 SQL。这套机制轻量、透明、AI 友好，运营同事也能在页面上自助改。

run-cron：定时任务独立进程入口，只启动调度器不启动 API，避免重任务阻塞线上接口。

这些 CLI 工具的设计理念是把高频运营操作封装成"安全、留痕、可重复"的命令，AI 用着高效，人审查也放心。AI 不用理解底层连接、事务这些细节，调一个命令就行。

6、前端组件化



前端组件化是老生常谈，但在 AI 编码场景下有一个特殊价值：**组件化让审查变得可行。**

如果一个 Vue 文件有 1000 行，AI 改了一处，人很难快速判断影响面。但如果拆成 100~200 行的子组件，每个组件职责单一，AI 的改动通常集中在一两个组件里，审查时只需看这几个文件。

我们的规则是：

- Vue SFC 控制在 500 行内，超过即拆
- 独立功能域抽子组件，可复用逻辑抽 composable，工具函数移 `utils/`
- 子组件 100~200 行，composable 30~70 行
- 所有组件配套 Storybook story

最后一条特别重要，目前项目里有 143 个 story 文件，覆盖了几乎所有组件。**Story 不只是给设计师看的，更是给 AI 和开发者提供的"组件说明书"**，这是因为 story 里定义了组件的

各种状态和 props 组合，AI 开发新功能时可以先看 story 了解已有组件能力，避免重复造轮子。开发完新组件后补 story，也相当于做了一次自测。

页面视图层走"智能组件 + 展示组件"模式。View 层只协调，把专线 ID、出口 ID、时间范围传给子组件，每个数据卡片自己 fetch、自己渲染。**View 层保持整洁，新增卡片不会让主文件膨胀。**

这套约束写在 AGENTS.md 里，AI 写前端时会自觉遵守。偶尔超了，提醒一句就拆好了。

7、让 AI 看见问题



AI 编码最大的风险不是写得慢，而是写错了不知道。让 AI 能"看见"问题是质量保障的关键。

我们的做法分四层：

第一层：静态检查即反馈。后端 ruff + ty check，前端 oxlint + vue-tsc，这些工具的输出 AI 都能直接读到。AI 写完代码后自己跑检查，有报错自己修。这比等人来 review 高效得多。

第二层：AGENTS.md 汇总规则。项目根目录的 AGENTS.md 集中了所有开发护栏和工程偏好，包括文件红线、分层规范、代码风格、DB 变更流程等。AI 每次会话都会读这个文件，相当于有一个"项目规范手册"随时可查。规则越集中，AI 遵守得越好。

第三层：开发服务 AI 自主管理。项目通过一个 `dev.sh` 脚本统一启动前后端开发服务，这个脚本本身就是 AI 写的，后续怎么改、日志在哪儿看、进程怎么查，全部交给 AI 管理。开发者不需要记这些细节，跟 AI 说一声"启动开发环境"或"看下后端日志"就行。AI 对开发环境的掌控力越强，自己发现和解决问题的能力就越强。

第四层：Playwright 验证。AI 开发完功能后用 Playwright headless 截图、检查 API 请求、验证交互行为。这相当于 AI 自己做了一遍 QA。

四层叠加，大部分问题 AI 会话内就能发现和修复，到人审查时已经比较成熟了。

8、DB 变更管控



DB 变更在生产系统里是最敏感的操作。我们设计了一套"极简、透明、可验证"的流程。

极简：所有 DDL/DML 通过 `flow-db-exec` 一个入口执行，AI 不需要写临时脚本。常用的加字段、加索引、按主键更新，直接执行后告知即可。

透明：任何变更必须先落到 changelog.sql（注明日期与目的），同步更新主结构定义文件，然后按行号执行刚追加的片段。这样变更历史可追溯，新环境重建数据库也有完整记录。

可验证：按风险分级处理。

操作	处理方式
加字段、加索引、按主键单行 UPDATE	直接执行后告知
ALTER 改字段类型/名/删字段	先讲方案，用户确认后再执行
不带主键的批量 UPDATE	先 SELECT COUNT(*) 评估，确认后执行
DROP / DELETE / TRUNCATE	硬拦截，改用软删除或人工执行

时间字段统一用 DATETIME，禁止 BIGINT 时间戳，方便运营同学理解。AI 在数据库操作上有清晰边界感，既高效又安全。

9、Agent 工作流



Agent 智能体是这个项目的重要能力。这里想重点分享的不是 Agent 框架的技术细节，而是**AI 原生的 Agent 研发方式**。

传统做法是：**开发者手动搭建 Agent 平台，在某个管理后台配置提示词、注册工具、设置参数，再把能力暴露出去。这种模式下，每新增一个 Agent 都需要人在多个系统间来回操作，维护成本高，迭代慢。**

> 说起来有点意思，我之前牵头搞过一个智能体平台，主打配置化，拖拽填表就能搭 Agent，上线比 Knot 还早。但现在 AI 写代码能力大幅提升之后，也许没必要整那么重，直接用代码描述 Agent：提示词是文件，工具是函数，注册是装饰器。**Everything as Code，这才是 AI 原生的研发流程。**

我们的做法完全不同：**给 AI 一个业务目标，让它自己基于现有代码仓库端到端地完成 Agent 的创建**。

具体来说，当我们需要一个新 Agent，例如"专线质量分析"，只需要跟 AI 说清楚这个 Agent 要解决什么业务问题、应该具备什么能力。AI 会自己完成全部工作：

- 探索代码仓库，理解现有的数据模型、服务接口和工具能力
- 编写系统提示词，定义 Agent 的行为规范和输出格式
- 复用或创建工具函数，把需要的数据查询和分析能力封装成 Agent 可调用的工具
- 在注册表中登记，接入统一的会话管理和流式输出框架
- 补充 Storybook story 做验证

整个过程不需要去任何外部系统手动配置，不需要在管理后台填表单，所有产物都在代码仓库里，可追踪、可 review、可回滚。

这里有一个更深层的设计理念需要强调：**系统中有什么能力，AI 自己去看**。项目的所有内容，包括数据模型、服务接口、工具函数、Agent 提示词、前端组件、定时任务等，都在同一个代码仓库里，AI 都可以获取到。这就是面向 AI 原生的设计：不把能力藏在某个管理后台或外部系统里，而是全部以代码形式存在，让 AI 能直接探索、理解、复用。

传统的智能体组织形式是 **"平台 + 后台配置"**：Agent 平台是中心，工具和提示词在管理后台维护，人要去后台注册和配置。这种模式下，AI 看不到全貌，每接一个新能力都需要人手动"告诉"平台。而**我们的做法是"代码即配置"：Agent 的提示词是代码文件，工具是代码函数，注册是代码装饰器。AI 需要什么能力，直接在代码仓库里找，找到了就能用。不需要任何中间环节**。

这种设计带来的好处是显而易见的：AI 新增一个 Agent 时，不是在白纸上画画，而是在一个已经充满能力的生态里"搭积木"，看到有现成的流量查询 service 就复用，看到有现成的图表渲染工具就直接接，看到有其他 Agent 的提示词写法就参考。整个过程的效率，远高于在管理后台从零开始配置。

这背后的支撑是我们在框架层做了统一抽像：一个 ReactAgent 基类负责模型配置、上下文注入、流式事件输出、会话持久化等通用能力。新增 Agent 只需关注三件事：叫什么名字、系统提示词是什么、有哪些工具，基类和注册机制把其余的都包了。

还有几个设计细节值得提一下：

1、Agent 通过 SSE 流式输出，支持断连重连，客户端刷新页面后能从上次断开的位置继续，跑几分钟的分析任务中途刷新不会丢失；

2、工具沉淀有一个重要约定：**工具返回值尽量是 markdown 字符串**而非原始 JSON，一来 Agent 更好理解，二来大幅减少 token 消耗，此外人在页面上也看起来更方便。

下图展示了 Agent 创建技能的部分内容，完整内容的可以去文末的原始代码仓库中查看：



AI 原生的 Agent 研发方式，核心价值就是：**人只关注业务价值**，即这个 Agent 要解决什么问题、输出什么结论。

至于提示词怎么写、工具怎么对接、框架怎么注册，全由 AI 端到端搞定。人的精力花在定义"对的问题"上，而不是"对的配置"上。

10、Anydev 统一研发环境

前面讲的 Rules、Skills、CLI 工具都是"AI 怎么干活"的护栏，但还有一个前提问题：**开发环境本身怎么准备好**。

传统模式下，一个新同事加入项目，光是准备开发环境就要折腾半天。比如装系统依赖、装 Python 和 Node 工具链、配置私有源认证、生成环境变量文件、装前后端依赖、启动开发服务。每一步都可能踩坑：系统包版本不对、私有源地址记不住、环境变量漏配导致服务启动报错。对非开发同事来说，这道门槛几乎不可逾越。

我们用一套自动化初始化脚本解决了这个问题。基于公司的 Anydev 云研发容器，开发环境在容器启动时自动完成全部配置，**从创建容器到可用，3 分钟以内，全程零人工介入。**



具体实现在 `scripts/system/setup.sh`，按顺序串起 7 个子步骤：

第一步：上报开发环境。容器启动后第一时间把自己的 IP、前后端服务地址、开发分支等信息注册到项目的 devops 管理页面。这样运营同事在网页上就能看到谁的开发环境在线、地址是什么，直接点链接就能验收功能，不用问"你的开发环境地址是多少"。

第二步：安装系统依赖。自动安装 mysql-devel、gcc、gettext 等编译和运行所需的系统包。考虑到容器内 dnf 偶发网络抖动，做了最多 3 次重试。

第三步：安装工具链。安装 uv（Python 包管理）和 pnpm（前端包管理），以及 rtk（内部研发工具）。装完立即 source 环境变量，让后续步骤的当前 shell 就能用。

第四步：注入开发规范。把 `.vscode/anydev_rule.md` 拷贝到 `.codebuddy/rules/` 目录。这样 AI 在容器里一启动就自动加载项目规范，不需要人手动配置，前面讲的三层 Rules 中的第二层就这么自动就位了。之所以做延迟加载，是为了区分平台基础能力开发、业务逻辑开发这两个场景，让专业开发不受限制。

第五步：渲染 private.env。项目的私密配置（七彩石 Token、Git 私有源认证、UV 私有源账号等）不能明文提交到代码仓库，我们用 `private.env.template` 模板 + `envsubst` 在容器启动时渲染生成。模板里用 `${VAR}` 占位，容器环境变量注入实际值。渲染前会校验所有必填变量，缺一个就直接报错退出，不会生成一个"半残"的配置文件让后续服务启动时才报莫名其妙的错。

第六步：安装项目依赖。后端 `uv sync`、前端 `pnpm install`，一条命令搞定。前端安装时一次性放行 `onlyBuiltDependencies` 的构建脚本，避免后续启动时因 ERR_PNPM_IGNORED_BUILDS 退出。

第七步：启动开发服务。调用 `dev.sh` 启动前后端开发服务，等待端口就绪后打印访问地址。`dev.sh` 本身也做了端口占用检查、进程管理、日志重定向，支持 `start/stop/restart/status` 四个命令。

这套脚本的设计有几个关键点：

一是**每步独立、可单独执行**。7 个步骤各自是独立的 shell 脚本，放在 `setup_steps/` 目录下。调试某个步骤时可以单独跑，不用每次从头来。

二是**失败即停、原因清晰**。每步都 `set -e`，任何一步失败立即中断，并打印具体的错误信息。比如 private.env 渲染时缺变量，会列出具体缺哪些变量名，而不是生成一个有问题的文件让后续步骤报含糊的错。

三是**幂等可重跑**。脚本设计为可重复执行，已经装过的不会重复装，已经在运行的服务不会重复启动。

这套机制的效果是：**任何同事，不管会不会写代码，从 Anydev 模板创建一个开发容器，等几分钟就能拿到一个完整可用的开发环境**。前后端服务已经启动，AI 已经加载了项目规范，私有配置已经就位，开发分支已经切好。打开 CodeBuddy 直接说需求就行。

这和前面讲的"通用底层能力"是配套的：通用底层能力是代码层面的基础设施，Anydev 统一研发环境是环境层面的基础设施。两者叠加，才让"AI 原生研发"对整个团队真正可用，不是"理论上能用"，而是"打开就能用"。

三、开发与产品协同

页面评论到自主开发



这是我们在"AI 原生研发"上最有代表性的实践：让不懂开发的网络运营同事也能直接提需求，AI 自主完成开发。

具体机制是这样的：

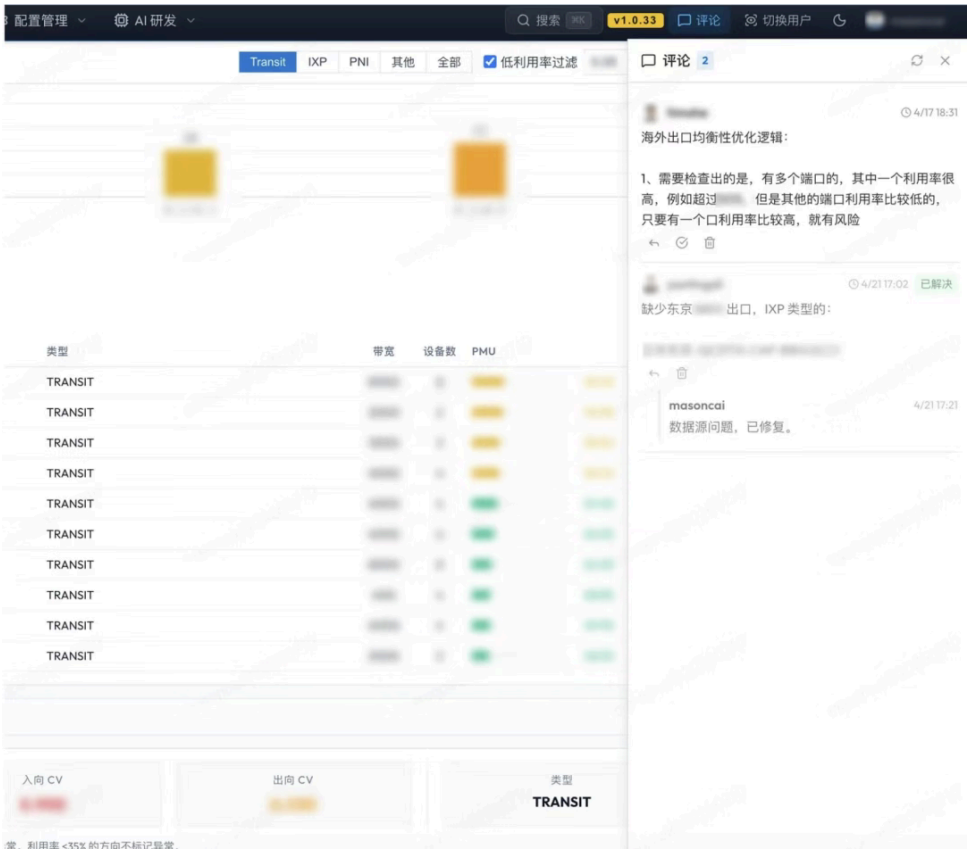
页面级评论系统。我们在每个页面植入了评论功能，运营同事可以在页面上任意位置圈点评论。评论会记录页面路劲、元素定位信息（xpath、html 片段、元素坐标），还能附带截图。这相当于一个"所见即所得"的需求提交工具。运营同事看到哪里有问题，直接在页面上标注，不需要写需求文档，不需要懂技术术语。

评论状态流转。评论有 open / resolved / closed 三种状态，开发同事确认后改为 resolved，上线后改为 closed。整个流程在页面上可见，运营同事能随时看到自己提的问题处理到哪了。

复杂需求走 SDD 流程。如果评论背后的需求比较复杂，开发同事会把评论内容整理成 `features/` 下的 `draft_` 文档，AI 读完文档后参与讨论，方案确定后改为 `ready_`，AI 自主开发，最后由提需求的人验收。

开发容器自助注册。每个开发同事有自己的开发容器，容器启动后自动上报环境信息（前后端地址、分支等）到 devops 管理页面。运营同事可以在开发容器上提前验收功能，确认没问题再合并上线。

一个例子：



核心价值是把需求提报门槛降到最低。运营同事不用学任何开发工具，页面上圈圈点点就能提需求。AI 把需求翻译成代码，开发同事负责 review 和兜底，每个人做自己最擅长的事。

四、Lessons from AI Coding

1、避免廉价习得感



用 AI 写代码久了，容易产生一种"我也会编程"的错觉。这种廉价习得感是危险的。

实际上，AI 写出来的代码如果人不理解，就没办法做好审查，质量就没办法保障。我们要求团队同学学习一些基本的开发术语和概念：什么是分层架构、什么是 ORM、什么是中间件、什么是 SSE。不是为了让大家都会写代码，而是为了能看懂 AI 写的东西，能给出有质量的反馈。

"这个函数职责太重了，拆一下"、"这个逻辑应该放 service 层不是 controller"、"这个 SQL 没走索引"，这些反馈比"感觉不对，你再看看"有用得多。**AI 的输出质量很大程度上取决于人的反馈质量，而人的反馈质量取决于对开发概念的理解程度。**

我们的经验是：**运营同事不需要会写代码，但需要能读懂代码的大致逻辑。花一点时间理解基本概念，会让 AI 协作的效率提升一个量级。**

2、给非开发者一张"能力地图"



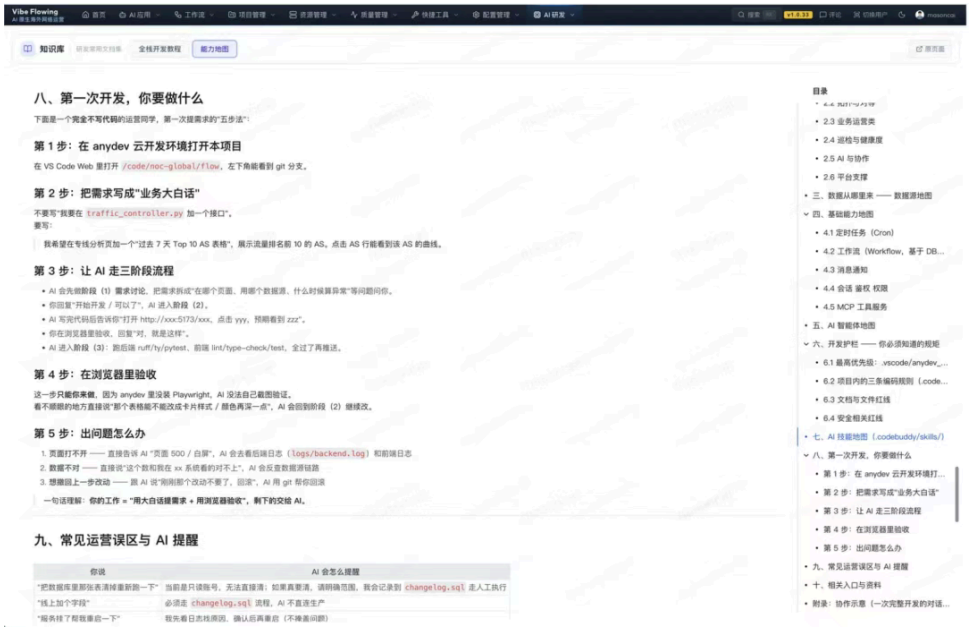
上面说的是开发侧要学概念，其实在用户侧，即使用这个系统的网络运营同事，同样面临"概念不清"的问题，只是维度不同。

运营同事不需要懂代码，但他们需要懂"产品语言"。比如页面上每个区域叫什么：导航栏、面包屑、侧边栏、主体内容区、弹窗、抽屉……这些在前端开发里是常识，但对运营

同事来说并不直观。如果一个人连"把侧边栏收起来看看"都听不懂，那和 AI 协作提需求时就会有很多沟通损耗。

更关键的是，运营同事需要知道**这个项目整体具备什么能力**。我们的系统有几十个页面、十几个 Agent、若干定时任务和开放 API，散落在各处。如果运营同事不知道系统已经能做什么，提需求时就会要么重复提（"能不能加个出口流量对比"，其实已经有了），要么不敢提（不知道这个系统能做这么复杂的事）。

我们的解决办法是做了一份**能力地图**，一份结构化的文档，把整个项目的功能模块、Agent 能力、定时任务、开放 API 按业务域分类罗列，每个能力附一句简要说明。运营同事读一遍，就能对"这个系统能做什么"有个整体认知。



成本很低，AI 几分钟就生成完了，但效果是显著的：读完能力地图后，运营同事提需求时明显更"有的放矢"了，大家知道哪些是已有能力的微调，哪些是真正的新功能；和 AI 对话时也能更准确地描述上下文（"在流量工作台页面的分光 TopN 卡片这里，我想加一个……"），AI 理解起来也更高效。

总的来说，**AI 原生研发要降低的门槛，不只是"写代码"，还有"理解产品"**。能力地图这类轻量文档投入不大，但对非开发者的参与度提升很明显。

3、省 token



最后聊一个工程性强但很重要的话题：如何省 token。

在 AI 原生研发模式下，token 消耗是实打实的成本。一次会话如果上下文太大，不仅贵，还慢。我们在实践中总结了几个有效的做法：

AGENTS.md 集中规则。项目规范写在一个文件里，AI 读一次就够了，不需要在每个文件里重复说明。规则集中也意味着 AI 不需要到处翻找约定。

工具返回 markdown 而非原始数据。Agent 工具返回格式化好的 markdown 表格或摘要，而不是原始 JSON。一来 Agent 更好理解，二来大幅减少 token 消耗，一个格式化表格可能 200 token，而原始未经整理的 JSON 可能得花 10 倍。

CLI 工具替代临时脚本。`flow-db-exec` 一行命令搞定的事，不需要 AI 写 50 行 Python 脚本去 import db、建连接、执行 SQL、格式化输出。命令的输出本身就是精简的。

大仓 + 语义搜索。单仓 monorepo 让 AI 一次会话能覆盖全栈，配合语义搜索快速定位相关代码，减少"到处找文件"的探索开销。

SDD 文档驱动。复杂需求写成文档，AI 读完文档就有完整上下文，不需要人在对话里反复补充信息。文档也比对话历史更精练。

组件化 + Storybook。AI 开发新功能时先看 story 了解已有组件，避免重复实现。组件拆得小，AI 每次改动的范围也小，上下文不需要加载整个大文件。

探索一些可行的工具。比如 RTK （Rust Token Killer）等技术，不过验证下来之后，发现效果一般，偶尔压缩时还会出 bug，一些压缩策略还会导致语义混淆，就不用了。绝大部分 Bash 命令都能通过参数来精确控制输出啥，这些 AI 抖动，没必要再整一个新的。

省不了几块钱，还让 AI 再依赖一个不稳定的命令行代理，得不偿失～

```
root@VM-34-35-tencentos [12:16:06] [/code/noc-global/flow] [dev]
-> # rtk gain
RTK Token Savings (Global Scope)
═══════════════════════════════════════════════

Total commands:    448
Input tokens:      146.4K
Output tokens:     38.7K
Tokens saved:      108.6K (74.2%)
Total exec time:   142m49s (avg 19.1s)
Efficiency meter: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 74.2%

[warn] No hook installed — run `rtk init -g` for automatic token savings

By Command
```

#	Command	Count	Saved	Avg%	Time	Impact
1.	rtk:toml ps aux	3	71.2K	97.4%	92ms	
2.	rtk git diff --cached	1	14.4K	69.5%	13ms	
3.	rtk env	4	6.5K	99.7%	0ms	
4.	rtk playwright test -...	1	5.4K	99.9%	4.1s	
5.	rtk find	2	3.7K	99.8%	7ms	
6.	rtk git log	1	1.2K	71.5%	4ms	
7.	rtk git log --since=2...	1	1.1K	71.2%	32ms	
8.	rtk pip list	1	1.0K	49.6%	1.0s	
9.	rtk playwright test /...	2	907	98.3%	2.2s	
10.	rtk ls -la /code/noc-...	1	638	56.6%	5ms	

```
root@VM-34-35-tencentos [12:16:09] [/code/noc-global/flow] [dev]
-> #
```

小结

说到底，AI 原生研发不是把所有东西丢给 AI，而是搭好一套让 AI 高效干活的基础设施和协作机制。基础设施越好，AI 产出质量越高，人的审查负担越轻。**这是个正向循环，值得在工程上持续投入。**

五、总结



回看全文，AI 原生研发的核心可以归纳为一句话：**搭好底座，让 AI 在上面高效干活，让团队里每个人，不管会不会写代码，都能参与建设。**

具体来说，我们做了三层事情：

第一层：基础设施。把日志、鉴权、权限控制、开放 API、定时任务、MCP 工具这些通用能力沉淀成标准设施，新项目直接继承。业务 SDK 以 submodule 形式集成，AI 自行探索理解能力。这一层的价值是：AI 开发新功能时不需要从零搭基础设施，也不会在底层问题上浪费 token。

第二层：工程护栏。大仓分层 + AGENTS.md 规范 + anydev_rule 流程约束 + Memory 记忆系统，构成 Rules 三层护栏；Agent 创建、前端设计、Vue 开发、工蜂等 Skills 预装技能包。配合 CLI 工具、DB 变更管控、TDD 验收闭环、SDD 文档驱动、组件化 + Storybook，让 AI 既守规矩又高效。这一层的价值是：AI 的行为有下限保障，人的审查负担降到最低。

第三层：协作机制。页面评论提需求、Anydev 统一开发模板、能力地图降低理解门槛，让网络运营同事不写代码也能参与建设。Agent 研发走 AI 原生路线，给目标，AI 端到端完成，网页直接可用。这一层的价值是：需求提报门槛降到最低，每个人做自己最擅长的事。

三层叠加，形成正向循环：基础设施越好 → AI 产出质量越高 → 人的审查负担越轻 → 人有更多精力优化基础设施。不是一蹴而就的事，是在实践中持续迭代的結果。

针对已有的存量项目，我们正在按照这趟方案，构建大仓、搭好 Harness 工程骨架，让更多业务也能参与到开发中来。

这套方案不是 AI 研发的终点，而是 AI 时代开发和产品协作的新的起点。，我们还在路上，但方向是明确的：**不是让 AI 替代人，而是搭好一套让 AI 和人都能高效发挥的体系，把整个团队带到下一个台阶，即「AI 原生的研发&运营团队」。**





阅读 3.0万

留言

写留言

还没有留言，来写首评